“Event Management System”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms - 3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

by

2520030385 – P. Tanvitha

2520030132 – N. Tejasri

2520030018 – P. Laxmikanth

Team Number : 14

Section: 6

Under the Guidance of

Dr. V. Sireesha

Associate Professor

Department of Computer Science and Engineering

2026–27



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

The project report entitled “Eventora: Event Discovery, Booking and Personalized Recommendations” is a record of bona fide work carried out by the undersigned students and submitted in partial fulfilment of the requirements for the course Data Structures and Algorithms - 3 (25CS2103E) in the Department of Computer Science and Engineering during the academic year 2026–27.

The implementation and discussion in this report are based on the project source code supplied in the eventora_favorites_added ZIP archive. The report describes the modules and algorithms present in that source. Any feature that requires an external service or production deployment is identified as a limitation rather than being represented as a verified live service.

We declare that this report has been prepared for academic submission and that all external tools, libraries and documentation referred to in the report are acknowledged in the References section.

2520030385 - P. Tanvitha

2520030132 – N. Tejasri

2520030018 – P. Laxmikanth

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the mini-project report entitled “Eventora: Event Discovery, Booking and Personalized Recommendations” is a bona fide work carried out and submitted by the students listed below in partial fulfilment of the requirements for the course Data Structures and Algorithms - 3 (25CS2103E), Department of Computer Science and Engineering, during the academic year 2026–27.

2520030385- P. Tanvitha

2520030132 -N. Tejasri

2520030018 – P. Laxmikanth

The report follows the prescribed chapter structure and formatting guidelines to the extent possible from the provided project archive. Student names, roll numbers, team number, section and signatures must be completed before final submission.

Signature of the Guide	**Signature of the HOD**
Dr. S. Vinay Kumar	Department of CSE

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his guidance, encouragement and valuable suggestions during the planning and development of this mini project. His guidance helped us organize the work into clear modules and consider the importance of algorithmic thinking, interface usability and systematic documentation.

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic environment and support required to carry out the project. We also thank the faculty members and our classmates for their feedback and encouragement throughout the development process.

Finally, we acknowledge the maintainers of React, Vite, React Router, Lucide React and the other open-source tools used by the project. Their documentation and software ecosystem helped us build a component-based web interface and organize the application into reusable pages and algorithm modules.

2520030385 – P. Tanvitha

2520030132 – N. Tejasri

2520030018 – P. Laxmikanth

ABSTRACT

Eventora is a web-based event discovery and booking interface designed to bring event exploration, event details, favourites, ticket information, calendar browsing and recommendation features into a single application. The submitted source code uses React for the user interface, Vite as the development and build tool, React Router for navigation and Lucide React for interface icons. Event records include a title, category, date, time, venue, price, description, image and featured status.

The application is organized into distinct page modules, including Home, Explore, Event Details, Booking, My Tickets, Favorites, Dashboard, Calendar, Create Event, Recommendations, Login and Register. Users can browse sample events, filter and search the catalogue, open event detail pages, select booking information, save favourites, view tickets and create custom event entries. The code stores several kinds of data in browser local storage, including favourites, custom events, user records and tickets. This makes the supplied version suitable as a client-side prototype, while also creating limitations around data persistence across devices, authentication security and real payment processing.

A key academic aspect of Eventora is the inclusion of separate algorithm modules. Pattern matching performs case-insensitive substring matching across event title, category and location. Fuzzy matching uses Levenshtein edit distance to tolerate small spelling differences. A token-set similarity function calculates an overlap ratio, and a ranking function scores events using featured status, favourites and title matches. These algorithms demonstrate how data structures and string-processing techniques can support event discovery and recommendation-oriented ordering.

The report explains the problem context, system requirements, architecture, module-wise implementation, algorithm pseudocode, observed implementation characteristics, limitations and possible future work. Results in this report are based on inspection of the supplied source code; no claims of measured response time, production payment completion or external-service integration are made unless supported by the archive.

TABLE OF CONTENTS

Section	Page No.
Acknowledgement	4
Abstract	5
Table of Contents	6
List of Figures	7
1. Introduction	8
1.1 Overview	8
1.2 Problem Statement	10
1.3 Scope of the Project	12
1.4 Objectives	14
2. System Requirements	15
2.1 Functional Requirements	15
2.2 Non-Functional Requirements	17
2.3 Hardware and Software Requirements	19
2.4 Background and Related Concepts	21
3. System Design	24
3.1 High-Level Architecture	24
3.2 Description of Each Layer	26
3.3 Data and Navigation Design	31
3.4 Algorithm Design	33
4. Implementation	38
4.1 Module-Wise Implementation	38
4.2 Front-End Logic and State Management	40
4.3 Data Persistence and Integration	42
4.4 Pseudocode	44
5. Results	49
5.1 Implemented Features	49
5.2 Algorithm Walkthroughs	50
5.3 Verification Checklist	52
6. Challenges and Limitations	55
6.1 Issues Faced During Development	55
6.2 Solutions Applied	57
6.3 Limitations of the System	58
7. Conclusion and Future Work	60
7.1 Conclusion	60
7.2 Future Work	62
7.3 Skills Learned During Development	64
References	65

Note: Page numbers should be refreshed after opening the document in Microsoft Word or LibreOffice, because pagination depends on the installed fonts and final student details.

LIST OF FIGURES

Figure No.	Caption
1.1	Conceptual view of the Eventora event discovery problem
3.1	High-level architecture of Eventora
3.2	Event discovery and booking navigation flow
3.3	Client-side data persistence flow
4.1	Algorithmic processing pipeline for event discovery
5.1	Feature coverage based on the supplied source archive

1. INTRODUCTION

1.1 Overview

Digital event platforms allow people to discover conferences, music festivals, workshops, sports activities, exhibitions and community gatherings. As the number of events grows, a user needs more than a static list: they need to search by relevant words, narrow results by category, compare event details, save interesting options and keep booking information organized. A well-structured event application can reduce the effort required to move between these tasks.

Eventora is a client-side web application that combines these activities in a single interface. The source archive includes a set of React pages connected through React Router. The Home page introduces the platform, Explore provides catalogue browsing, Event Details presents an individual event, Booking collects booking details, My Tickets displays stored tickets, Favorites lists saved events, Calendar provides date-oriented browsing, Create Event allows custom event entry, and Recommendations presents a recommendation-oriented view. Login and Register pages are also included.

The sample catalogue in the source code contains eight event records covering Technology, Music, Business, Design, Sports, Arts and Education. Each record contains fields such as id, title, category, date, time, location, price, description, image and featured. The data is therefore structured enough to support display, filtering, search, sorting and event-detail navigation. The project also contains independent algorithm files for similarity, fuzzy matching, pattern matching and ranking.

The implementation is best understood as an academic prototype rather than a production ticketing service. Several user-facing activities are implemented using browser local storage, which keeps information in the current browser profile. The supplied archive contains an Express-related backend package manifest, but the reviewed route setup and data modules primarily show a React/Vite front end; a complete live backend API or payment gateway cannot be confirmed from the supplied files.

Fig. 1.1 presents the main user problem and the role of Eventora as an integrated client-side interface.

User need	Fragmented workflow problem	Eventora response
Find relevant events	Users may need to scan many unrelated listings.	Explore page with search, category and sorting controls.
Understand an event	Important details can be spread across multiple views.	Event Details page with date, venue, description and price fields.
Keep a shortlist	Interesting events can be difficult to revisit.	Favorites page and browser storage for saved favourites.
Organize attendance	Booking and ticket details may be difficult to track.	Booking flow and My Tickets page.
Discover alternatives	Exact text search may miss close spellings or related titles.	Separate fuzzy-match, pattern-match, similarity and ranking utilities.

Fig. 1.1 Conceptual view of the Eventora event discovery problem

The growth of online platforms has changed the way communities organize and attend events. A decade ago, announcements for concerts, seminars and sports meets were circulated through posters, printed passes and word of mouth, and attendance depended on whether a person happened to see the right notice at the right time. Today the same information is published through websites and mobile applications, which makes the supply of events much larger but also places a new burden on the attendee: the person must filter a large amount of material, judge which listings are relevant, and then complete a sequence of separate steps before an event can actually be attended. Eventora is positioned as an attempt to reduce that burden, within the limits of a course project, by placing discovery, shortlisting, booking and ticket review in a single and consistent interface.

The decision to build Eventora as a React single-page application follows from the nature of the task. In a single-page application the browser loads the application once, and later navigation is handled by JavaScript instead of by requesting a new document from a server for every click. For an event platform this is useful because the user moves repeatedly between the same few views, namely the catalogue, a detail page, a booking form and a ticket list, and each transition should feel immediate. React supports this style by describing the interface as a tree of components whose output is a function of their state, so that when a search term, a category or a favourite flag changes, only the affected parts of the screen are drawn again. Vite supplies the development server and the production build, while React Router maps address-bar paths to page components so that each view can be reached through a meaningful path.

Within the context of Data Structures and Algorithms - 3, the interface is only one half of the project. The more academic half lies in the way event records are represented, searched and ordered. An event is modelled as an object with a fixed set of fields, the catalogue is an array of such objects, favourites are a collection of identifiers, and search is performed by algorithms that operate on strings and sets. Each of these choices corresponds to a data-structure decision that can be analysed: arrays provide ordered storage and linear traversal, objects provide keyed access to fields, sets provide duplicate-free membership testing, and a two-dimensional matrix supports the dynamic-programming computation of edit distance. By implementing these structures inside a working application rather than in isolation, the project connects abstract course material to behaviour that a user can observe on the screen.

This report is organized in seven chapters. The remainder of this chapter states the problem, defines the scope and lists the objectives. Chapter 2 describes the functional and non-functional requirements, the environment needed to run the application and the background concepts on which the algorithms rely. Chapter 3 presents the architecture, the data design and the design of each algorithm, and Chapter 4 explains the implementation of the individual modules together with pseudocode. Chapter 5 reports the results in terms of implemented features, worked examples and a verification checklist, Chapter 6 discusses challenges and limitations, and Chapter 7 concludes with future work and the skills gained. Throughout the report, statements about the system are restricted to what can be established from the supplied source archive, and features that depend on external services are described as limitations rather than as achievements.

1.2 Problem Statement

The problem addressed by Eventora is the effort involved in browsing event listings and managing the information needed to attend them. A basic event list may expose event names but provide limited support for finding a particular category, handling spelling variation, retaining favourites or organizing booking records. When these tasks are separated, users may have to repeat searches and manually remember event details.

The project therefore aims to provide a unified interface in which event information is represented consistently and can be accessed through several task-oriented pages. It also demonstrates how string-processing and ranking algorithms can improve the way event records are found and ordered. The central technical problem is to connect a structured event catalogue, navigable React components, client-side state and lightweight search/recommendation logic into one coherent application.

The source code does not establish that the system is connected to a verified real-world event provider, payment processor or remote user database. Accordingly, the problem statement is limited to the implementation available in the ZIP archive: a browser-based prototype with sample event data and local persistence.

The difficulty faced by an attendee can be described as three separate kinds of friction. The first is discovery friction: when a catalogue contains events from many categories, a person looking for one workshop or one match has to scan unrelated entries, and a plain list offers little help in narrowing the field. The second is retention friction: an event that looks interesting today may be forgotten tomorrow unless the platform provides a convenient place to keep a shortlist. The third is completion friction: after an event has been chosen, the person must supply details, confirm a quantity, choose a payment option and later retrieve proof of the booking. When these three stages are handled by unconnected tools, the person repeats effort and may lose information between steps.

From a technical point of view, the problem can be divided into four sub-problems. The first is representation, which asks how an event should be described so that every page of the application can use the same record. The second is retrieval, which asks how a typed query should be compared with those records so that relevant events are returned even when the query is incomplete or slightly misspelled. The third is ordering, which asks how the matching events should be arranged so that the most useful ones appear first. The fourth is persistence, which asks how favourites, tickets and user-created events can survive a page refresh without requiring a database server. Eventora addresses each of these four sub-problems with a small and explainable technique, and the interaction between the techniques forms the core of the project.

Exact search deserves particular attention because it is the technique most often assumed to be sufficient. A strict match requires the typed characters to appear in the stored text exactly as entered, and it fails as soon as a single character is wrong. A user who types a close but incorrect spelling of a word in an event title receives an empty result and may wrongly conclude that no such event exists. Case sensitivity is another source of avoidable failure, because a query written in lowercase would not match a title that begins with a capital letter. The project therefore treats tolerant matching as a requirement of the problem and not as an optional extra, and it uses lowercasing, substring matching, edit distance and token overlap as complementary tools.

Finally, the problem statement must be read together with the constraints of a course project. The team does not have access to a verified event provider, a payment processor or a hosted database, and the report must not imply otherwise. The problem is therefore defined as the construction of a coherent browser-based prototype whose behaviour can be inspected in the supplied source code, whose algorithms can be explained step by step, and whose limitations are stated openly so that a reader can distinguish what has been implemented from what would be needed for a real service.

1.3 Scope of the Project

The project scope includes front-end event discovery, event detail presentation, favourites, booking form flow, ticket display, calendar-based exploration, custom event creation, a recommendation page, and basic login/registration interfaces. It also includes reusable UI components such as event cards, category cards, ticket cards, search bars and navigation. Algorithm scope includes case-insensitive pattern matching, approximate matching based on edit distance, token-set similarity and a weighted ranking procedure.

The data scope is the set of eight sample events defined in the event data module together with custom events stored in local storage by the browser. The application can merge stored custom events with the built-in event list. The exact list may be extended by the user through the Create Event interface.

Out of scope for the confirmed implementation are secure production authentication, cross-device synchronization, authoritative event inventory, real

ticket issuance by a venue, a verified payment transaction, email/SMS notifications, and a deployed server-side database. These can be considered future enhancements, but should not be described as completed functions.

The functional scope of the project is organized around a single kind of user who wishes to find and attend events. The application does not distinguish between an organizer account and an attendee account at the level confirmed by the source; instead, any visitor can browse the catalogue, mark favourites, complete the booking interface and use the Create Event form to add a custom record. This simplification keeps the prototype small, and it also explains why the Login and Register pages are treated as interface demonstrations: they show how an account flow would be presented, but they do not establish a role-based permission model.

The algorithmic scope has been limited deliberately to techniques that are lightweight, deterministic and easy to explain. Pattern matching, Levenshtein-based fuzzy matching, token-set similarity and weighted ranking can all be described in a few sentences, traced by hand on a small example and implemented in a few dozen lines of JavaScript. This choice makes the algorithms suitable for an academic report, because every step can be justified. It also means that the project does not attempt machine-learning recommendation, semantic search, geographic search or personalised scoring from behavioural history, all of which would require larger datasets and an evaluation procedure that is beyond the present scope.

The environmental scope is the web browser. The application is intended to be opened in a current desktop or mobile browser after the dependencies have been installed and the development server or production build has been started. All data handled by the application lives in the memory of the page or in the local storage of the browser profile. As a consequence, the behaviour of the system on a second device, in a private browsing window or after the stored data has been cleared is different from its behaviour in the original browser, and these differences are part of the documented scope.

The exclusions listed above are not an admission of neglect but a recognition of what a front-end prototype can honestly claim. Secure authentication requires server-side password hashing and session management. Authoritative inventory requires a database that is shared by every user. A verified payment requires a

gateway, server-side confirmation and protection of sensitive data. Notifications require a messaging service. By recording these exclusions in the scope, the report ensures that later chapters can describe them as future work and that no reader mistakes the booking and ticket screens for a live commercial service.

1.4 Objectives

- To develop a single web interface for browsing and discovering event information.
- To organize the application into reusable React pages and components.
- To provide search, category filtering and sorting options for event exploration.
- To allow users to mark events as favourites and revisit their saved list.
- To provide booking and ticket-management interfaces backed by client-side data.
- To support user-created event records through a custom-event form.
- To demonstrate pattern matching, fuzzy matching, similarity scoring and ranking algorithms.
- To apply route-based navigation so that pages have clear, direct paths.
- To document the implementation honestly, including assumptions and limitations visible in the source.

The objectives listed above can be grouped into interface objectives, data objectives, algorithm objectives and reporting objectives. The interface objectives concern the first, second, third and eighth items, which together call for a single web interface made of reusable React pages and components, equipped with search, category filtering and sorting, and connected by route-based navigation. Meeting these objectives means that a user can reach each function through a stable path, that common elements such as event cards and the navigation bar look and behave consistently, and that adding a new page would require only a new component and a new route entry.

The data objectives concern favourites, tickets and custom events. The application should allow a user to mark an event as a favourite and find it again later, to complete the booking interface and see the resulting ticket record, and to create a new event record that appears alongside the sample events. These objectives are met through browser local storage, which is sufficient for demonstration but not

for production use. It is therefore important that the objectives are phrased in terms of client-side data, as they are in the list, because a stronger statement would not be supported by the source archive.

The algorithm objective is the most directly related to the course. It asks the project to demonstrate pattern matching, fuzzy matching, similarity scoring and ranking. The demonstration is considered successful when each algorithm exists as an independent function, when its behaviour can be explained in terms of the underlying data structure, and when its role in event discovery is clear. Each algorithm is analysed in Chapter 3, written as pseudocode in Chapter 4 and traced on small examples in Chapter 5, so that the objective is addressed at the levels of design, implementation and illustration.

The final objective, honest documentation, is easy to overlook but is treated as a design principle of the whole report. Every chapter distinguishes between behaviour that can be read directly from the source and behaviour that would need external services or measurement. Quantitative claims about speed, accuracy or booking success are avoided unless the team has measured them, and the verification checklist in Chapter 5 is presented as a procedure to be followed rather than as a record of completed tests. In this way the objectives remain measurable: each one can be checked against the source archive or against a defined observation in the running application.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

Functional requirements describe the observable tasks the application is designed to support. The following requirements are derived from the pages, route definitions and local-storage use in the supplied source archive.

A requirement is useful only when it can be traced to something that the system actually does and tested by an observation that can be repeated. For this reason each functional requirement in Eventora is stated in terms of a page or a function that exists in the source archive, and each one carries a short identifier so that it can be referred to in later chapters. The identifiers FR-01 to FR-14 are used in the verification checklist and in the feature inventory, which allows a reader to follow

a requirement from its statement to its implementation and then to the test that would confirm it.

ID	Requirement	Source-based description
FR-01	Home page	Provide an entry page that introduces the event platform and directs users to event exploration.
FR-02	Explore events	Display event cards and allow catalogue browsing.
FR-03	Search	Accept a search query and support event discovery using text-oriented logic.
FR-04	Filter and sort	Allow event lists to be narrowed by category and sorted by available options.
FR-05	Event details	Open an event-specific page using its route identifier.
FR-06	Favourites	Save and retrieve favourite event records from browser local storage.
FR-07	Booking	Collect booking details and store a ticket/booking record in browser storage.
FR-08	My Tickets	Display locally stored ticket records.
FR-09	Calendar	Provide a calendar-oriented view of event dates and selected-day exploration.
FR-10	Create event	Accept custom event details and store the new record locally.
FR-11	Recommendations	Provide a recommendation-oriented page with category/like interactions.
FR-12	Account pages	Provide registration and login interfaces using browser-stored user records.
FR-13	Routing	Connect pages through React Router paths.
FR-14	Algorithm utilities	Provide pattern matching, fuzzy matching, similarity and ranking functions.

The fourteen requirements fall naturally into five groups. FR-01 to FR-04 form the discovery group and describe the entry page, the catalogue, the search box and the filtering and sorting controls. Together they define how a user moves from arrival to a manageable list of relevant events. The requirement for search is deliberately phrased in terms of text-oriented logic and not of one particular algorithm, because the interface can use exact matching, fuzzy matching or ranking without changing what the user is entitled to expect. The requirement for filtering and sorting similarly allows several category and ordering options to exist behind a common control.

FR-05 and FR-06 form the inspection and shortlisting group. Opening an event through its route identifier means that the detail page receives only the identifier from the address and must find the matching record in the combined catalogue of sample and custom events. The favourites requirement adds a persistent action to this page and to the catalogue, which is why the same storage key is read by more than one page. These two requirements illustrate the principle that a shared data

structure and a shared storage key keep different views of the application consistent without a central server.

FR-07 and FR-08 form the transaction group. The booking requirement covers the collection of details such as quantity, payment choice, name, email, phone and a UPI identifier, the creation of a ticket record and the transfer of the user to the ticket page. The ticket requirement is its counterpart: it reads the stored records and presents them. The requirement is written as storing a record in browser storage, not as issuing a ticket, because the source does not confirm any exchange with a venue or a payment provider. This wording avoids overstating the function while still describing a complete user-visible flow.

FR-09, FR-10 and FR-11 form the planning and contribution group. The calendar offers a date-oriented way of finding events, the Create Event page lets a user add a record to the catalogue, and the recommendation page offers a different entry point based on categories and liked events. Finally, FR-12, FR-13 and FR-14 are supporting requirements: the account pages, the central route table and the algorithm utilities. They are less visible to the user than the other groups, but they are the reason the application can be extended, because they define the boundaries between interface, navigation and logic.

Each requirement also implies an input, a processing step and an output. For search, the input is the typed query, the processing step is a comparison with event fields, and the output is a reduced or reordered list. For booking, the input is the form content, the processing step is the creation and storage of a record, and the output is a navigation to the ticket page and a visible ticket. Stating requirements in this form makes it straightforward to design test cases, because each test needs only to supply the input and compare the observed output with the expected one, as described for the verification checklist in Chapter 5.

2.2 Non-Functional Requirements

Non-functional requirements concern qualities such as usability, maintainability, responsiveness, reliability and security. Some are design goals rather than verified test results; the source archive does not include a formal performance test report.

Non-functional requirements describe how well the system performs its functions rather than which functions it performs. They are harder to specify than functional requirements because they depend on the context of use, and for a prototype many of them can be stated only as design goals. The table below records the qualities considered most relevant to Eventora, and the paragraphs that follow explain how each quality is supported by the design and which parts of it would still require measurement or additional work.

Quality	Requirement / assessment
Usability	Navigation, event cards, page-specific forms and recognizable icons should make common actions understandable.
Maintainability	Pages and algorithm utilities are separated into individual files; repeated interface elements are placed in components.
Responsiveness	The application includes CSS files for layout and page presentation. Responsiveness should be verified on multiple viewport sizes before submission.
Performance	Local sample data avoids the need for a network request for the built-in event catalogue. No measured response-time figures are available.
Reliability	Parsing of some local-storage values is guarded with try/catch. Browser storage can still be cleared, corrupted or unavailable.
Security	Client-side local storage is not a secure store for passwords, payment data or trusted booking records. Production deployment requires server-side validation and authentication.
Scalability	A local in-browser catalogue is appropriate for a prototype but is not a substitute for a database and API when users and events grow.
Compatibility	The project uses modern JavaScript, React and Vite; target browsers and versions should be tested as part of deployment.

Usability is supported mainly by consistency. Event information is shown through the same card layout on several pages, the navigation bar offers the principal sections from every page, and icons from the Lucide library accompany actions so that a user can recognise controls without reading long labels. Usability also depends on feedback: a user who marks a favourite or completes a booking should see a visible change, and a notification component exists in the interface for this purpose. A formal usability study with real participants has not been carried out, so the claim is limited to the presence of design features that are generally associated with good usability.

Maintainability is the quality most clearly demonstrated by the source organization. Each page lives in its own file, shared elements are placed in components, the route table is centralised, and the four algorithms are separated from the pages that use them. A developer who wants to change the ranking weights needs to edit only the ranking file, and a developer who wants to add a new page needs to add one component and one route entry. This low coupling is

also what makes the algorithms testable in isolation, since they are pure functions that accept arguments and return values without touching the page.

Responsiveness and performance are related but distinct. Responsiveness concerns whether the layout adapts to different screen widths, and the presence of dedicated style sheets suggests that layout was considered, although multi-device verification is still required. Performance concerns the time taken to react to user actions. With a catalogue of only eight sample events plus any custom events, even a linear scan with an edit-distance computation for every word completes far faster than a person can perceive, so the absence of a measured figure does not signal a problem at the present scale. At larger scales the situation would change, and the algorithms would need indexing, caching or server-side execution.

Reliability, security, scalability and compatibility form the group in which the prototype is weakest, and the report treats them candidly. Reliability is improved by guarded parsing of stored values, which prevents a corrupted entry from stopping the page, but local storage can still be cleared or blocked. Security is limited because browser storage is readable and writable by any script running on the same origin and by the user through developer tools, and it must never be treated as a trusted store for credentials or payment data. Scalability is bounded by the fact that the catalogue lives in the front-end bundle, and compatibility is expected across modern browsers but has not been tested against a documented list of versions.

2.3 Hardware and Software Requirements

The environment needed to work with Eventora can be considered at two levels: the environment of a developer who installs the dependencies, edits the source and builds the application, and the environment of an end user who only opens the finished application in a browser. The developer environment is the more demanding of the two, and it is the one described in the table. The end-user environment is much lighter, since the built application consists of static files that any modern browser can execute.

Type	Minimum / recommended environment
Hardware	Computer capable of running a current web browser and Node.js development tools; keyboard, pointing device and internet access for installing dependencies.

Operating system	Windows, Linux or macOS with a supported Node.js runtime.
Runtime	Node.js and npm, with versions compatible with the package versions in package.json and package-lock.json.
Front-end framework	React and React DOM.
Build tool / dev server	Vite.
Routing	React Router DOM.
Icons	Lucide React.
Styling	CSS files in src/App.css, src/index.css and src/event-pages.css.
Optional backend dependencies	The archive includes a backend package manifest listing Express, CORS and dotenv, but the reviewed files do not establish a complete running API service.
Browser	A current Chromium-based browser, Firefox or another modern browser supporting ES modules and local storage.

A developer needs a computer that can comfortably run a code editor, a terminal and a browser at the same time. The project does not demand special processing power, because the tools involved are written for ordinary laptops, but a reasonable amount of memory and free disk space is needed to hold the installed packages. The dependency folder created by the package manager is typically much larger than the project's own source files, which is normal for JavaScript projects and is the reason such folders are excluded when a project is shared as an archive. Internet access is required once, at installation time, so that the packages named in the manifest can be downloaded.

Node.js provides the runtime in which the build tools execute, and npm is the package manager that reads the manifest, resolves the dependency versions recorded in the lock file and installs them. The scripts declared in the manifest are conventionally started with commands such as npm run dev for the development server, npm run build for a production build, npm run lint for static analysis and npm run preview for serving the built output locally. The lock file is important for reproducibility, because it ensures that two developers who install the project obtain the same package versions, which reduces the chance of a problem that appears on only one machine.

Vite serves two roles. During development it runs a server that delivers source modules to the browser on request and updates the page quickly when a file is saved, which shortens the cycle between editing and observing. For production it bundles and optimises the code into static assets that can be hosted on any web server. React and React DOM provide the component model and the rendering of components into the page, React Router DOM provides client-side routing, and

Lucide React supplies icons as components. The style sheets are ordinary CSS files, which keeps the styling layer free of additional tooling.

The optional backend dependencies deserve a careful reading. Express, CORS and dotenv are the packages usually associated with a small Node.js web service that accepts requests from a browser application, allows cross-origin access and reads configuration from environment files. Their presence in a manifest shows that a back end was planned or begun, but the manifest alone does not prove that a server is implemented. The report therefore lists them as optional and does not require them for running the front end. Anyone who wishes to extend the project with a server will find that the manifest provides a starting point.

For the end user, the requirement is simply a browser that supports modern JavaScript modules and the local storage interface. Current versions of Chromium-based browsers and Firefox satisfy this, as do other recent browsers. Because the application stores data in the browser profile, users should be aware that private browsing sessions may discard the data when the window is closed, and that clearing site data removes favourites, tickets, custom events and user records. This behaviour is a consequence of the storage choice and should be explained to anyone who demonstrates the application.

2.4 Background and Related Concepts

Event discovery systems commonly combine catalogue browsing with search and filtering. A catalogue record is easier to process when it uses consistent fields, such as a unique identifier, title, category, date, location and price. In Eventora, the event data module defines this structure directly in JavaScript objects. This is sufficient for a small prototype and makes each event easy to render as a card or detailed page.

Exact substring matching is simple and predictable, but users may enter different capitalization or partial terms. Case-insensitive matching reduces capitalization sensitivity. Fuzzy matching goes further by estimating how many character edits separate a query from a word. The Levenshtein distance is a standard dynamic-programming measure of insertion, deletion and substitution operations. The

supplied implementation uses this distance to accept words within a query-length-based threshold.

Similarity scoring can help estimate whether two pieces of text share vocabulary. The project's similarity utility lowercases each input, splits it into whitespace-separated tokens, converts the tokens into sets and divides the size of their intersection by the size of their union. This is a Jaccard-style token similarity. The ranking utility is a weighted rule-based sorter: featured events, favourites and title matches contribute fixed score increments.

These techniques are lightweight and interpretable, but they are not equivalent to a trained recommendation model. The current score does not learn from user history, event attendance, ratings or collaborative behavior. A future version could compare multiple event fields, track explicit user preferences and evaluate recommendations with a defined test dataset.

The data structures that underlie the application are the ones introduced in the earlier parts of the course. An array holds the catalogue and preserves the order of insertion, which is useful when a list must be displayed or sorted. An object holds the fields of a single event and provides access by key, so that a title, a date or a price can be read directly. A set, created from the tokens of a string, stores only distinct values and allows an efficient check of membership and intersection. A two-dimensional matrix of integers stores the intermediate results of the edit-distance computation. Choosing the appropriate structure for each purpose is the practical lesson that the project is intended to reinforce.

String matching is a classical topic with a large literature. The simplest approach, naive substring search, slides the pattern over the text and compares characters at each position, and in the worst case it takes time proportional to the product of the lengths of the text and the pattern. More advanced algorithms such as Knuth-Morris-Pratt and Boyer-Moore reduce the work by reusing information from earlier comparisons. In the project, the substring test is delegated to the built-in string operation of the language, which is adequate for short fields such as titles and locations. The relevant point for the course is not the speed of one particular call, but the understanding that the choice of matching strategy affects both the cost and the kind of query that can be answered.

Approximate string matching relaxes the requirement that the pattern appear exactly. The Levenshtein distance between two strings is the minimum number of single-character insertions, deletions and substitutions needed to turn one into the other. It is computed with dynamic programming: a table is created whose entry at row i and column j holds the distance between the first i characters of one string and the first j characters of the other. The first row and column are filled with consecutive integers, representing insertions and deletions against an empty prefix. Every other entry is the smallest of three values: the entry above plus one, the entry to the left plus one, and the diagonal entry plus zero if the two characters are equal or plus one if they differ. The bottom-right entry is the answer, and the computation takes time and space proportional to the product of the two lengths.

Set-based similarity measures how much two collections have in common. The Jaccard index divides the size of the intersection of two sets by the size of their union, and it yields a value between zero and one, where zero means that the sets share nothing and one means that they are identical. Applied to the words of two texts, it gives a simple measure of vocabulary overlap that ignores word order and repetition. The measure is related to ideas in information retrieval, where documents and queries are compared through the terms they contain, although the full retrieval models used in search engines add term weighting, normalisation and indexing that the project does not attempt.

A ranking function assigns a numerical score to each item and orders the items by that score. When the score is computed from hand-chosen rules and fixed weights, the function is called a heuristic, and its behaviour is completely predictable. This predictability is an advantage for explanation and debugging, because the reason for any position in the list can be traced to the rules. The disadvantage is that the weights reflect the designer's assumptions and are not adjusted by evidence. Learned ranking models, by contrast, estimate their parameters from data such as clicks and bookings, but they require a large amount of data and a method of evaluation, and they are more difficult to explain.

The web storage interface offered by browsers provides a simple key-value store for strings. Data saved through it belongs to a particular origin, which means that pages from a different site cannot read it through normal means, and it persists until it is removed by the script or the user. Because only strings can be stored,

structured data such as arrays of events must be converted with JSON serialisation before saving and parsed after reading. The interface is synchronous, which is acceptable for small amounts of data, and its capacity is limited to a few megabytes. These properties make it convenient for prototypes and unsuitable for sensitive or large datasets.

Finally, the React library introduces concepts that the report refers to repeatedly. A component is a function that returns a description of part of the interface. State is data that a component remembers between renders and that, when changed through the setter function, causes the component to render again. Hooks such as useState and useEffect allow function components to hold state and to perform side effects such as writing to storage. React Router adds the idea of a route, which associates a path with a component and allows parts of the path to be read as parameters. Together, these ideas allow the interface to be built from small, composable pieces whose behaviour follows from their inputs.

3. SYSTEM DESIGN

3.1 High-Level Architecture

Eventora follows a client-side component architecture. The browser loads the React application, the route layer selects the appropriate page, page components read and update event-related state, and shared components present events, categories, tickets and navigation. The algorithm utilities provide independent functions that can be called by the relevant discovery logic. Browser local storage provides persistence for selected records between page visits in the same browser profile.

Layer	Main elements	Responsibility
Presentation	Home, Explore, EventDetails, Booking, MyTickets, Favorites, Dashboard, Calendar, CreateEvent, Recommendation, Login, Register	Render the screens and handle user interactions.
Reusable components	Navbar, EventCard, CategoryCard, SearchBar, TicketCard, Notification	Reuse common visual and interaction patterns.
Application routing	AppRoutes.jsx and React Router	Map URL paths to page components and provide parameterized event/booking paths.
Domain data	src/data/events.js	Supply built-in event records and combine them with locally stored custom events.

Algorithm utilities	similarity.js, fuzzySearch.js, patternMatching.js, ranking.js	Perform token similarity, approximate matching, substring matching and weighted ordering.
Browser persistence	localStorage keys used by pages	Retain favourites, custom events, user records and ticket records in the current browser.
Build environment	React, Vite, npm scripts	Run the development server, build static assets and run linting.

Fig. 3.1 High-level architecture of Eventora

The architecture can be understood by following a single user action through the layers. Suppose a user types a word into the search box on the Explore page. The SearchBar component reports the new text to the page, the page stores it in its state, and React renders the page again. During that render the page obtains the combined event list from the data layer, applies the matching logic from the algorithm layer, perhaps orders the result, and returns a list of EventCard elements. Nothing in this sequence requires a request to a server, which is why the response to a keystroke is immediate. The same pattern repeats for category selection, sorting and favourite toggling, with only the state variable and the function in the algorithm layer changing.

The separation of the system into layers brings several benefits that are relevant to a project of this size. Because the presentation layer does not contain the matching algorithms, a page can be changed without risking a change in search behaviour. Because the data layer is reached through a single function that combines built-in and custom events, a page does not need to know where the events come from. Because routing is declared in one place, the set of reachable pages can be read from a single file. These separations reduce coupling, which is the degree to which a change in one part forces a change in another, and low coupling is a widely accepted characteristic of maintainable software.

The architecture also has deliberate gaps. There is no server layer in the confirmed design, no shared database and no authentication service. Persistence is delegated to the browser, and every user effectively operates a private copy of the application. This simplifies installation, since nothing needs to be deployed apart from static files, but it prevents shared features such as common inventory, organizer dashboards that reflect real attendance, or the same ticket being visible on several devices. The figure of the architecture should therefore be read as a

map of what exists, and the future-work chapter explains where a server tier would be added.

A final observation concerns the build environment, which appears as a layer in the table although it is not part of the running application. The inclusion is intentional, because the way the project is built and checked influences its quality. The scripts for development, building, linting and previewing allow the code to be run, packaged and inspected consistently, and the linting step in particular encourages a uniform code style and the early detection of unused variables or suspicious constructs. A reliable build process is a precondition for the testing and deployment steps recommended later in the report.

3.2 Description of Each Layer

The architecture of Eventora can be described layer by layer, from the part that the user sees to the part that stores data. The five layers described in this section are the presentation layer, the routing layer, the data layer, the algorithm layer and the persistence layer. They are not isolated from each other: the presentation layer calls on the data and algorithm layers to prepare what it shows, the routing layer decides which presentation component is active, and the persistence layer supplies the stored values that the data layer and several pages read. Describing them separately, however, makes it easier to explain the responsibility of each and to identify what would have to change if one of them were replaced.

A useful way of reading the description is to ask two questions of each layer: what does it know, and what does it deliberately not know? The presentation layer knows about screens, controls and styling but not about how a string is compared with another. The algorithm layer knows about strings, sets and scores but not about buttons. The data layer knows the shape of an event but not how it is displayed. This discipline, in which every layer has a narrow field of knowledge, is the main reason the application stays understandable as the number of pages grows.

3.2.1 Presentation Layer

The presentation layer consists of React function components. Each page is responsible for a user task, while shared components avoid duplicating common UI. For example, event cards can present event metadata consistently in multiple

pages, and the navigation component offers routes to the principal application sections. CSS files provide layout, spacing, typography, colors and responsive styling.

Each page component acts as a controller for one task. The Explore page, for example, keeps the information needed for browsing, namely the query, the selected category, the sort mode, the visibility of the filter panel, the favourites and the custom events, and it combines them to decide which cards to display. The Booking page keeps the values typed into the form and the chosen payment option. The Calendar page keeps the selected date. This pattern, in which each page owns the state that concerns its own task, avoids a large global state and keeps the data flow easy to follow.

The reusable components make the interface consistent. An event card that shows an image, a title, a date, a location and a price is needed on the Explore page, the Favorites page and possibly the Calendar and Recommendation pages, and writing it once guarantees that these pages present the same event in the same way. The navigation component provides the same links on every page, the search bar provides the same input behaviour, and the ticket card presents ticket information in a form that can be repeated for as many tickets as the user holds. Consistent presentation reduces the effort a user needs to learn the interface.

Styling is delivered through ordinary style sheets. A global sheet establishes the base appearance, the application sheet governs the main layout, and a dedicated sheet for the event pages controls the presentation of cards, forms and ticket views. Keeping styling in separate files makes it possible to adjust the visual identity without editing the logic of the components. Responsive behaviour, such as rearranging a grid of cards on a narrow screen, is normally obtained through flexible layout rules and media queries, and the report recommends checking these rules at several viewport sizes before final submission.

3.2.2 Routing Layer

The route configuration declares a parent route at the application root and nested paths for the Home, Explore, Event Details, Booking, Tickets, Favorites, Dashboard, Calendar, Create Event, Recommendations, Login and Register pages. The event and booking routes include an identifier parameter. A wildcard route

returns the Home page for unmatched paths, providing a simple fallback rather than a dedicated not-found page.

Routing allows the address shown in the browser to describe the part of the application that is active. A parent route at the root of the application typically renders elements that are common to all pages, such as the navigation bar, and the nested routes render the page-specific content inside it. This arrangement means that moving from one page to another replaces only the page content and leaves the shared frame in place, which is both efficient and visually stable.

Parameterised routes are essential for event-specific pages. A path that contains an identifier segment allows one component to serve every event: the component reads the identifier from the address, looks up the matching record and renders it. The same technique is used for the booking path, so that a booking is always attached to a particular event. A major advantage is that such addresses can be bookmarked or shared, and a refresh of the page returns to the same event because the identifier is part of the address and not only of the in-memory state.

The wildcard route is a small but meaningful design decision. When a user enters an address that does not match any declared route, the application returns the Home page instead of an error. This avoids a blank screen and keeps the user inside the application, but it also hides mistakes, since a mistyped address and a deliberate visit to the home page look the same. A dedicated not-found page that explains the problem and offers links to the main sections would be a worthwhile refinement and is listed among the improvements that can be made without changing the architecture.

3.2.3 Data Layer

The built-in data module exports an event array and a getAllEvents function. The function attempts to parse the customEvents value from local storage and then returns locally created events together with the built-in events. A try/catch block falls back to an empty custom-event list if parsing fails. Each event record contains data needed for listing and detail presentation.

The data layer is intentionally small. It exports an array of eight sample events and a function that returns the complete list of events by adding any custom events found in local storage. Because every page that needs events obtains them through

this function, the rest of the application does not need to know whether an event is built-in or user-created. A custom event is distinguished only by the way it arrived, and it carries the same fields as a sample event, so it can be displayed, searched, favourited and booked with no special treatment.

The guarded parsing in the function shows an important principle of defensive programming. The stored text is converted back into an array with a parsing operation that raises an error if the text is not valid, for instance if it has been edited manually or truncated. By catching the error and falling back to an empty list, the function ensures that the built-in events remain available and that the page continues to work. The price of this safety is that corrupted custom events are silently ignored, so a more complete design would also log the problem or offer the user a way of recovering the data.

The sample events are static, which means that their dates, prices and descriptions do not change unless the source is edited. This is acceptable for demonstration, but it also means that the catalogue cannot reflect real availability. Any future integration with a real provider would replace the array with data fetched from an interface, and the function that combines the lists would become asynchronous. Because pages already depend on a single function for events, this migration would affect the data layer more than the pages, which is a benefit of having placed the data access behind one entry point.

3.2.4 Algorithm Layer

The algorithm layer separates text-processing functions from the React page components. The similarity function calculates overlap between unique tokens. The fuzzyMatch function first handles an empty query, then tests for a direct case-insensitive substring, and finally checks individual words using Levenshtein distance. The pattern-matching functions search event title, category and location. The ranking function sorts a copied event array by a weighted score based on featured status, favourites and title matches.

The algorithm layer consists of four files, each exporting functions with a clear contract. The pattern-matching functions accept text and a pattern, or a list of events and a pattern, and return a Boolean or a filtered list. The fuzzy-matching function accepts a query and a text and returns a Boolean. The similarity function

accepts two strings and returns a number between zero and one. The ranking function accepts events, a query and favourites and returns a sorted copy of the events. None of these functions reads from or writes to the screen or storage, which makes them examples of pure functions: for the same arguments, they return the same result and cause no side effects.

Purity has practical advantages. A pure function can be tested by calling it with chosen inputs and comparing the output with the expected value, with no need to render a page or prepare storage. It can be reused by any component that needs it, and it can be replaced by a different implementation, for example a more efficient edit-distance routine, without any change to the callers as long as the contract is respected. The project takes advantage of this by keeping the algorithms in a dedicated directory, and the report recommends that unit tests be written for them as the first step of verification.

The layer also illustrates the idea of composing simple tools. The pattern-matching functions provide a fast and strict filter, the fuzzy matcher provides tolerance, the similarity function provides a graded measure of closeness and the ranking function provides ordering. A search experience could combine them by first narrowing the list with a tolerant test, then ranking the survivors, and optionally using similarity to find related events. The source archive makes each tool available, and the combination to be used on each page is a choice that can be adjusted without changing the algorithms themselves.

3.2.5 Persistence Layer

The application uses browser local storage for selected user-facing records. The inspected pages reference keys including eventoraFavorites, customEvents, users and tickets. This persistence is convenient for a demonstration because it does not require database setup. However, it is limited to the browser profile and is controlled by the user; it does not provide multi-user synchronization or secure server-side storage.

The persistence layer stores four kinds of record under four keys. The favourites are stored as a list of event identifiers or event records under a dedicated key, the custom events are stored as an array of event objects, the registered users are stored as an array, and the tickets are stored as an array of booking records. All

four are converted to text with JSON serialisation before being saved and are parsed after being read. The choice of one key per kind of record keeps the code simple and allows each page to read and write only what it needs.

A particular characteristic of this approach is that the pages are responsible for the consistency of the data. There is no schema, no transaction and no constraint, so nothing prevents two pages from writing differently shaped records under the same key, and nothing prevents a record from referring to an event that no longer exists. In a small prototype this risk is manageable, but it is the reason that serious applications move persistence to a database, which can enforce structure, relationships and access control.

Another characteristic is that the storage is shared by all tabs of the same browser profile but is not automatically synchronised in the state of the pages. If a user has the application open in two tabs and adds a favourite in one, the other tab will show the change only when it reads the storage again, for instance after a reload. The browser can notify other tabs of storage changes through an event, and a refinement of the application could listen for it. Awareness of this behaviour helps explain some surprising results during testing, and it is one more reason for defining a clear procedure for the verification of favourites.

3.3 Data and Navigation Design

Field	Purpose	Example from supplied event data
id	Identifies the event and supports route lookup.	1
title	Human-readable event name.	Hyderabad Tech Summit
category	Used for grouping and category filtering.	Technology
date	Event date in year-month-day format.	2026-09-28
time	Display time for the event.	10:00 AM
location	Venue or location text.	HICC, Hyderabad
price	Numeric sample price.	499
description	Summary shown to the user.	A technology conference description.
image	Image URL used for event presentation.	Unsplash image URL
featured	Boolean flag used by ranking logic.	true

The main navigation flow begins at the Home page and continues to Explore. Selecting an event opens the parameterized Event Details route. From there, the user can enter the Booking route and later inspect locally stored ticket records. Favourites are managed separately and can be reopened through the Favorites route. Custom events are created through the Create Event page and become part of the catalogue when the data module reads the saved records.

Fig. 3.2 Event discovery and booking navigation flow

Stage	Input	Output / next stage
1. Browse	Open Home or Explore.	Event catalogue is displayed.
2. Search/filter	Enter query or select category/sort option.	Catalogue is narrowed or reordered.
3. Inspect	Select an event card.	Event-specific details page opens.
4. Shortlist	Toggle favourite control.	Favourite list is updated in browser storage.
5. Book	Enter booking information and quantity.	A local ticket/booking record is stored by the booking flow.
6. Review	Open My Tickets.	Stored ticket records are displayed.

The data design begins with the observation that every field in the event record serves a specific purpose, as the table shows. The identifier supports lookup and routing and must be unique, which is simple to guarantee for the built-in events but requires care for custom events, since a newly generated identifier must not collide with an existing one. The title, location and category are text fields that take part in search. The category also takes part in filtering, and for this to work reliably the category names should come from a controlled list, because a spelling variation such as an extra letter would create a separate group.

The date field uses the year-month-day format, which has a useful property: when dates are written in this format, comparing them as strings gives the same order as comparing them chronologically, so sorting by date does not require conversion. A word of caution applies when such a string is converted into a date object, because in some environments a date-only string is interpreted as a moment at midnight in universal time, and the displayed day may shift in a different time zone. A careful implementation either avoids unnecessary conversion or specifies the time zone explicitly. The time field, in contrast, is a display string such as 10:00 AM and is not designed for arithmetic.

The price is stored as a number, which allows sorting and, where a booking interface needs it, the computation of a total from a quantity and a unit price. A numeric price also makes it possible to format the value consistently, with a

currency symbol applied only at the moment of display. The image field holds a web address, so the picture itself is not part of the project files; this keeps the archive small but means that the displayed image depends on the external address remaining available. The featured flag is a Boolean that the ranking function reads, and it represents a promotional decision about which events deserve priority.

Navigation design follows the sequence in the second table. A user starts by browsing, refines the list by searching or filtering, opens an event, may shortlist it, books it and later reviews the tickets. The sequence is not strictly linear, since a user can open the Favorites page directly, return to Explore from any page or create an event at any time. What the sequence provides is a main path, which the design makes short and obvious: from the Home page to a ticket, a user passes through a small number of screens, each of which offers a clear next action. A short main path is a recognised principle of interaction design, because it limits the number of decisions between intention and result.

The data design and the navigation design depend on each other. The event identifier is the thread that connects the catalogue, the detail page, the favourites list and the ticket records, and the route parameter is the means by which that thread is carried through the address. If the identifier were missing or duplicated, several features would fail at once. The design therefore treats the identifier as the most important field of the record, and any extension of the application, such as importing events from a provider, must preserve its uniqueness and stability.

3.4 Algorithm Design

The algorithms of Eventora are designed to be small, deterministic and explainable. Each one addresses a different weakness of simple text search, and the four together form a pipeline that can take a typed query and a list of events and produce an ordered list of results. Pattern matching provides the strict baseline, fuzzy matching adds tolerance for spelling mistakes, token similarity provides a graded comparison between two texts, and ranking decides the order in which the surviving events should be shown.

For every algorithm, the discussion below states the input and the output, describes the steps, analyses the cost and identifies the cases in which the behaviour might surprise a user. Cost is expressed with the usual asymptotic notation, in which n is the number of events and m and k are the lengths of strings. Since the sample catalogue is small, these costs are not a practical concern at present, but analysing them is part of the discipline of the course and shows how each technique would behave as the catalogue grows.

3.4.1 Pattern Matching

The containsPattern function returns false when either the text or pattern is empty. Otherwise, it converts both values to lowercase and checks whether the text includes the pattern. The findMatches function returns the full event list for a blank pattern; for a non-blank pattern it filters events by concatenated title, category and location. This design makes matching case-insensitive and straightforward to explain.

The input to the pattern matching function is a piece of text and a pattern, and the output is true or false. The decision to return false when either value is empty at the function level, while returning the complete list when the pattern is blank at the list level, reflects two different questions. The first asks whether an empty pattern occurs in a text, to which the answer is not meaningful in a search context. The second asks what a user expects to see when the search box is empty, and the natural answer is that no filter is applied. Separating the two cases prevents a blank query from hiding the whole catalogue.

The fields that are concatenated for the match are the title, the category and the location. Concatenation means that a single comparison covers all three, which is efficient to write and to run, but it also means that a pattern could match across the boundary between two fields if the joined string contains the pattern's characters in sequence. For example, the end of the title and the beginning of the category could together form the pattern. Inserting a separator character between the fields would prevent such accidental matches, and it is a small improvement worth considering. The description field is not included, so a word that appears only in the description will not produce a match.

The cost of the filter is proportional to the number of events multiplied by the cost of one substring search. Using the naive approach, one search takes time proportional to the product of the lengths of the text and the pattern in the worst case, but with short fields and short queries the practical cost is very small. The extra space is proportional to the number of matches, since the result list holds references to the events that satisfy the pattern. These properties make pattern matching a good first step, and a reasonable design for a larger catalogue would keep it as a fast pre-filter before more expensive comparisons.

3.4.2 Fuzzy Matching

The fuzzy matching module calculates Levenshtein distance with a two-dimensional matrix. The first row and first column represent the cost of converting an empty string to a prefix through insertions or deletions. Each subsequent cell chooses the minimum of deletion, insertion and substitution cost. A query is accepted when it occurs directly in the text or when at least one word falls within a tolerance of max(1, floor(query length / 3)).

The fuzzy matcher is organised as a cascade of increasingly tolerant tests. A blank query is accepted at once, because there is nothing to compare. A direct case-insensitive substring match is accepted next, because it is the cheapest test and identifies the most common situation. Only if both tests fail does the function split the text into words and compute the edit distance between the query and each word. This order is a form of short-circuit evaluation: the expensive computation is performed only when the cheap checks have not produced an answer, which saves time in the typical case.

The tolerance rule, the larger of one and the floor of one third of the query length, scales the allowed number of errors with the length of the query. A query of up to five characters is allowed one edit, a query of six to eight characters is allowed two, and a query of nine to eleven characters is allowed three. The rule is sensible in principle, because a longer word offers more opportunity for typing errors, but it has consequences. For very short queries, one edit represents a large fraction of the word, so unrelated words such as short titles may be accepted. For longer queries, three or more edits may allow words that differ noticeably. The threshold

is therefore a heuristic whose value could be tuned against examples of real queries.

The computation of the distance fills a matrix with one more row and one more column than the lengths of the two strings. Each cell is derived from three neighbours, so the work for one pair of strings is proportional to the product of their lengths, and the space is the same unless only the previous row is retained, in which case it can be reduced to the length of the shorter string. For a catalogue of n events whose texts contain an average of w words, a query of length k and words of average length l, the total work is proportional to n times w times k times l, which remains small for the present data.

An important detail concerns the type of edit operations. Levenshtein distance counts insertions, deletions and substitutions, each at cost one, but it does not treat the swapping of two neighbouring characters as a single edit. A transposition such as typing the letters of a word in the wrong order therefore counts as two edits. A variant known as the Damerau-Levenshtein distance adds the transposition as a basic operation, and it can be more forgiving for typing errors. The limitation is illustrated in the walkthroughs of Chapter 5 and is mentioned among the possible improvements.

3.4.3 Token Similarity

The similarity function creates two lowercase token sets. It calculates the number of shared unique words and divides it by the number of unique words across both inputs. If the union is empty, it returns zero. Because the function uses sets, repeated words do not increase the score. Punctuation is not removed, so tokens with punctuation can remain distinct.

The similarity function treats each text as a bag of distinct words. By lowercasing, splitting on whitespace and building a set, it discards both the order of the words and the number of times each word occurs. The result is a measure that responds to shared vocabulary and nothing else. Two titles that contain the same words in different orders obtain a similarity of one, and a title that repeats a word many times is not rewarded for the repetition. These properties are desirable for comparing short descriptions, where order carries little additional information.

The value lies between zero and one. A value of zero means that the two texts share no words, and a value of one means that their sets of words are the same. Intermediate values show partial overlap, so that the function can answer the question of how close two event titles or descriptions are. This graded answer is more informative than a Boolean, and it can be used to order candidates, to find events related to a chosen event or to decide whether two records are likely to describe the same thing.

The cost of the function is proportional to the total number of words in the two inputs, since each word is inserted into a set once and the intersection is found by checking the members of one set against the other. The behaviour with punctuation illustrates a limitation of simple tokenisation. Because only whitespace is used as a separator, a word followed by a comma is a different token from the same word without the comma, and the overlap is underestimated. Removing punctuation before splitting would improve the result. Similarly, the function does not recognise that two different words, such as a singular and a plural form, have the same root, and stemming or lemmatisation would be needed to capture that.

3.4.4 Weighted Ranking

The rankEvents function sorts a copy of the event array so that the caller's original array is not directly sorted in place. Each featured event receives two points; an event present in the favourites list receives three points; and a title containing the non-empty query receives four points. Events are sorted in descending score order. This is a deterministic heuristic and not a learned ranking model.

The ranking function combines three signals into one score. Featured status represents the judgement of the platform, favourite membership represents the preference of the user, and a title match represents the relevance of the event to the current query. The weights of two, three and four give the strongest influence to relevance and the weakest to promotion, which is a reasonable ordering of priorities: when a user types a query, the events that match it should come before events that are merely promoted. The weights are, however, a design choice, and different weights would give a different ordering.

The function copies the array before sorting. This is important because the sort operation in JavaScript changes the array on which it is called. Sorting the original array would silently alter the order that other parts of the application rely on, and it would also violate the rule that state in React should be treated as immutable. By working on a copy, the function leaves its argument unchanged and returns a new array, which is the safer pattern. The sort is also stable in current JavaScript engines, so events with equal scores keep their original relative order.

The cost of ranking is the cost of computing a score for each event plus the cost of sorting. The scoring takes time proportional to n multiplied by the cost of the favourite check and the title check, and the sort takes time proportional to n log n comparisons. If the favourites are held as a list, each membership test takes time proportional to the length of that list, and holding them in a set would make the test constant in expectation. For the present scale none of this matters, but it is the type of consideration that the course asks students to make explicit.

The ranking function is a transparent heuristic, and the same transparency exposes its limits. It cannot rank two non-featured events differently if neither is a favourite and neither title contains the query, so ties are common. It does not use the event date, so an event that has already taken place can outrank a future one. It does not consider the category preferences of the user, the price or the distance. Each of these could be added as a new signal with its own weight, and the result could be evaluated against labelled examples, which is described in the future work section.

4. IMPLEMENTATION

4.1 Module-Wise Implementation

Module	Implementation summary
Home.jsx	Entry point for the event platform and links into the catalogue.
Explore.jsx	Maintains search, category, filter visibility, sort selection, favourites and custom-event state.
EventDetails.jsx	Uses the route event identifier to present the selected event and related actions.
Booking.jsx	Collects booking form values such as quantity, payment choice, name, email, phone and UPI ID; stores a ticket record in local storage and navigates to the tickets page.
Payment.jsx	Contains a payment-oriented interface and state for payment method and payment fields. The archive alone does not verify a real payment gateway or bank transaction.

MyTickets.jsx	Reads ticket records from local storage and presents upcoming/past or expanded ticket information.
Favorites.jsx	Reads and writes the eventoraFavorites key, with search and category state for managing the saved list.
Calendar.jsx	Provides a calendar interface with a selected date and search state for date-oriented event browsing.
CreateEvent.jsx	Collects custom event fields and stores created events in customEvents.
Recommendation.jsx	Provides a recommendation-oriented screen with category and liked-event state.
Login.jsx and Register.jsx	Provide account forms and use a users array in local storage; this is a prototype approach, not secure authentication.
Dashboard.jsx	Provides a user-facing dashboard view.
AppRoutes.jsx	Declares route-to-page mappings, including parameterized event and booking paths.
algorithms/*.js	Encapsulates matching, similarity and ranking operations.

The modules in the table can be grouped by the part of the user journey they serve. Home and Explore belong to discovery, EventDetails belongs to inspection, Favorites belongs to shortlisting, Booking, Payment and MyTickets belong to the transaction, Calendar and Recommendation offer alternative ways to find events, CreateEvent allows contribution, Login and Register provide the account interface, and Dashboard offers a summary view. AppRoutes ties these modules together and the algorithm files provide logic that the pages can call. Seeing the modules as a journey explains why the application feels coherent even though it is divided into many files.

The Explore module is the most state-rich page. It keeps the search text, the selected category, whether the filter panel is open, the chosen sort mode, the favourites and the custom events, and it derives the visible list from them. Because the visible list is computed from the state during rendering, there is no separate copy of the filtered results to keep in step. When the user changes any control, the state changes, the page renders again and the new list appears. This derivation of the display from a small set of values is the idiom that React encourages, and it prevents a class of errors in which two copies of the data disagree.

The Booking module demonstrates how a multi-field form is handled. The values for quantity, payment choice, name, email, phone and UPI identifier are held in state and updated as the user types, so that the form is controlled by React. When the user submits, the module reads the existing ticket records from storage, appends a new record and writes the array back, and then navigates to the ticket

page. The Payment module contains the corresponding interface for choosing a method and entering fields, and, as already noted, the archive does not show a connection to a payment provider, so it should be read as a presentation of the flow and not as proof of a completed transaction.

The MyTickets module reads the stored records and presents them, possibly separated into upcoming and past entries or shown in expanded form. The Favorites module is the counterpart for saved events, with its own search and category state so that a long list of favourites can be narrowed in the same way as the catalogue. The Calendar module maintains a selected date and offers date-oriented browsing, and the Recommendation module holds a category selection and a set of liked events. Dashboard provides a user-oriented overview. Together these modules make the application appear as a platform and not as a single page.

The Login and Register modules illustrate the account flow by storing and reading an array of users in the browser. They are appropriate for showing form layout, validation messages and navigation after sign-in, but they cannot protect an account, because anyone with access to the browser can read the stored array. This is why the report consistently describes them as a prototype approach. In a production system, the same screens would remain, but the logic behind them would be replaced by calls to a server that verifies credentials against hashed passwords and issues a session token.

4.2 Front-End Logic and State Management

React useState is used to keep interactive values such as the current search text, selected category, sorting mode, quantity, form fields and active calendar date. Pages derive the visible interface from this state. For example, Explore keeps the search query and category selection in state while managing favourites and custom events through local storage. This approach allows controls to update the view without a full page reload.

The Favorites page initializes its state by parsing the saved eventoraFavorites value and uses an effect to keep the browser storage updated when the list changes. Booking collects form values and, when the booking action is submitted, reads existing ticket records, adds a new record and navigates to My Tickets.

Create Event reads existing custom events and adds the submitted event to the stored array. These operations are local to the current browser.

React Router supports links between screens and provides route parameters for selecting a particular event. This creates a clear division between navigation and page content. Shared components such as EventCard and SearchBar support reuse and reduce repeated interface code.

State in React is local to the component that declares it, and the useState hook returns a pair consisting of the current value and a function to change it. When the function is called, React schedules a new render of the component with the updated value. The pages of Eventora use this mechanism for everything that a user can change through the interface. For text inputs, the value displayed in the field is taken from state and the change handler writes each keystroke back to state, which is the controlled-input pattern. The advantage is that the application always knows the exact content of each field and can use it for filtering or validation without reading it from the page.

The useEffect hook is used to synchronise the component with something outside React, and in this project that something is the browser storage. In the Favorites page, the state is initialised by reading and parsing the stored list, and an effect writes the list back whenever it changes. This arrangement makes the storage a mirror of the state: the user interacts only with the state, and the effect takes care of persistence. A point worth noting is that the effect runs after the render, so the storage is updated slightly after the screen, which is not noticeable in practice but is the reason that the initial value must be read at the time of the first render and not inside the effect.

Handling events consists of attaching functions to elements, such as a function that toggles a favourite when a heart icon is pressed. A correct handler never changes the existing array in place; it creates a new array, for example by filtering out an item or by adding an item to a copy, and passes it to the state setter. React compares the new value with the old to decide whether to render again, and an array that has been modified in place would be considered unchanged. The pseudocode for saving a favourite, described later, follows this rule by producing a new list in each case and storing it.

Navigation is performed with the tools of React Router. Link elements and navigation functions change the address without reloading the page, and the hook that reads route parameters allows the detail and booking pages to find the identifier of the event. After a booking is stored, the page calls a navigation function to move to the ticket page, and after a custom event is saved, the page returns to Explore. These transitions are triggered by events in the same way as other interactions, which means that the logic of a page, from reading the form to leaving it, can be read as a sequence of handlers.

Several refinements are possible. Values that are expensive to compute, such as a ranked list for a large catalogue, can be memoised so that they are recalculated only when their inputs change. State that is needed by several pages, such as favourites, could be lifted into a context so that all components share one source of truth, which would also remove the need for several pages to read the same storage key. Form validation could be centralised in helper functions, and error messages could be shown beside the fields. None of these changes alters the architecture, and each could be introduced step by step.

4.3 Data Persistence and Integration

The primary data source visible in the archive is a JavaScript array of sample events. Custom events are read from local storage and appended to the built-in list. Favourites, ticket records and account records are also stored under browser local-storage keys. This makes the prototype usable without a database server, but it means that records can disappear if browser data is cleared and are not automatically available on another device.

The root package declares development scripts for Vite, a production build, linting and previewing the built site. The dependencies include React, React DOM, React Router DOM and Lucide React. A separate backend package manifest lists Express, CORS and dotenv, but the supplied archive's visible route setup is front-end routing; no confirmed server route implementation or API contract was identified from the files reviewed. For an accurate project submission, this report therefore does not claim that a remote API is fully integrated.

The booking and payment pages should be interpreted as a user-interface flow unless the project team can demonstrate a live payment-provider integration

outside the ZIP. Real payments require a payment gateway, server-side verification, secure handling of transaction state and appropriate protection of sensitive data. Card details and authentication secrets should not be stored as plain local-storage values in a production application.

The conversion between in-memory arrays and stored text is a central part of the persistence logic. When a list is saved, it is passed through a serialisation function that produces a text in a standard interchange format, and when it is loaded, a parsing function reconstructs the list. The parsing function is the point at which things can go wrong, since text that is not in the expected format raises an error. The guards in the data and page modules catch such errors and substitute an empty list, which keeps the application running. The decision to guard parsing is a sign of attention to reliability, and it is recommended that every place that reads from storage follow the same pattern.

The lifecycle of a stored record is worth following. A favourite is created when the user toggles the control, read whenever the Explore or Favorites page is initialised, and removed when the user toggles it again. A custom event is created in the form, stored at the start of the array, read by the data function whenever the catalogue is built and persists until the storage is cleared. A ticket is created at booking and read by the ticket page. A user record is created at registration and read at login. None of these records has an expiry, a version or a link to an owner, which is acceptable for a demonstration but would have to be revised for a multi-user service.

Changes to the structure of stored records create a compatibility problem. If a later version of the application adds a field to the ticket record, tickets saved by the earlier version will not contain it, and the new code must either tolerate its absence or migrate old records. A simple practice is to store a version number with the data and write a function that upgrades older versions when they are read. For a prototype this may seem unnecessary, but it becomes important as soon as other people use the application over time and keep data in their browsers.

Integration with a real server would be done by replacing the storage calls with requests to an interface. A server would expose routes for events, users, favourites and bookings, validate every request, store the data in a database and return structured responses. The front end would then fetch events when a page loads,

show a loading state while waiting and an error message if the request fails, and send bookings to the server for confirmation. Because the present pages access data through a small number of functions, the substitution can be done gradually, starting with events and continuing with favourites and bookings.

The treatment of payment illustrates why integration cannot be trivial. A real payment involves the redirection of the user to a hosted page or the use of a secure component supplied by the provider, the creation of a payment intent on the server, the receipt of an asynchronous confirmation and the handling of failure, cancellation and repeated attempts. The confirmation must be verified on the server, not trusted from the browser, and the booking must be marked as paid only after verification. Sensitive details must not pass through the application's own storage. The report therefore recommends that payment be introduced first in a sandbox mode and that the booking record be given an explicit status field to record its progress.

4.4 Pseudocode

Pseudocode expresses the logic of an algorithm in structured, language-independent steps. It is more precise than a verbal description and less detailed than source code, and it allows the idea of an algorithm to be reviewed without the noise of syntax. The pseudocode in this section follows the behaviour of the functions in the source archive and of the storage operations performed by the pages. For each algorithm, a short commentary after the steps records its complexity, its assumptions and points that deserve attention when the algorithm is implemented or tested.

4.4.1 Event Pattern Search

Algorithm: FindMatches(events, pattern)

1. If pattern is blank after trimming, return events.
2. Create an empty result list.
3. For each event, combine its title, category and location into searchable text.
4. Convert the searchable text and pattern to lowercase.
5. If the searchable text contains the pattern, append the event to the result list.
6. Return the result list.

The algorithm begins with a guard clause that returns the entire list for a blank pattern. Guard clauses at the start of a function make the special cases explicit and keep the main logic free of nested conditions. After the guard, the loop visits each event exactly once, so the number of iterations equals the number of events. The creation of the searchable text by joining three fields is done for each event and can be seen as the preparation of a key against which the pattern is tested. The time required is proportional to the number of events multiplied by the cost of one substring test, and the space is proportional to the number of matching events.

When the algorithm is implemented, it is important to convert both the text and the pattern to the same case before comparison, since mixed case is the cause of most false mismatches. It is also worth trimming the pattern so that accidental leading or trailing spaces do not prevent a match, which the pseudocode reflects by testing for a blank pattern after trimming. A tester should check the behaviour for an empty query, a query containing only spaces, a query in upper case, a query that matches only the location and a query that matches nothing.

4.4.2 Fuzzy Matching Using Levenshtein Distance

Algorithm: FuzzyMatch(query, text)

1. If query is blank after trimming, return true.
2. Convert query to lowercase and trim surrounding whitespace; convert text to lowercase.
3. If text contains query, return true.
4. Split text into whitespace-separated words.
5. For each word, compute Levenshtein distance between query and word using dynamic programming.
6. If distance is at most max(1, floor(query length / 3)), return true.
7. If no word satisfies the threshold, return false.

The pseudocode of the fuzzy matcher represents a cascade of decisions. The first two checks are cheap and handle the blank query and the exact substring. The third stage splits the text into words, which restricts comparison to whole words and avoids comparing the query with arbitrary slices of the text. The distance computation itself is the standard dynamic-programming procedure and is usually

written as a separate helper function, so that it can be tested alone with pairs of strings whose distance is known.

The threshold formula is the most sensitive part of the algorithm. Because it is calculated from the length of the query, a change in the query length changes the tolerance at once, so tests should include queries on both sides of the length boundaries at which the threshold increases, namely lengths five and six and lengths eight and nine. The worst-case time is proportional to the number of words in the text multiplied by the product of the query length and the average word length. If performance ever became an issue, an early exit could be added that abandons the distance computation as soon as every value in a row exceeds the threshold.

4.4.3 Token Similarity

Algorithm: Similarity(a, b)

1. Convert each input to lowercase and split on whitespace.
2. Convert each token sequence into a set of unique words.
3. Calculate intersection = words common to both sets.
4. Calculate union = all unique words from both sets.
5. If union is empty, return 0; otherwise return size(intersection) / size(union).

The similarity algorithm is the simplest of the four, and its correctness follows directly from set theory. The intersection contains the elements present in both sets, the union contains the elements present in at least one, and their ratio is the Jaccard index. The guard for an empty union prevents a division by zero, which would otherwise occur when both inputs are blank. Returning zero in this case is a convention, since two empty texts could equally be regarded as identical, and the choice should be documented because it affects how results are interpreted.

The expected cost is linear in the number of words, because building a set and testing membership are constant-time operations on average. To test the function, one should use examples whose answer can be computed by hand, such as two texts sharing two of four distinct words, which should return one half, and texts with repeated words, which should return the same value as texts without repetition. A test containing punctuation shows the limitation of splitting only on

whitespace, and its expected result should be recorded so that any later improvement can be recognised as a change in behaviour.

4.4.4 Event Ranking

Algorithm: RankEvents(events, query, favorites)

1. Copy the events array so that the input order is not directly mutated.
2. For every event, initialize its score to 2 if featured, otherwise 0.
3. Add 3 if the event identifier occurs in the favourites list.
4. If query is non-empty and the title contains query without regard to case, add 4.
5. Sort events by descending score and return the sorted copy.

The ranking algorithm computes a score for each event as the sum of at most three contributions and then sorts by score in descending order. The upper bound on the score is nine, reached when an event is featured, is a favourite and has a title that contains the query. The procedure begins by copying the array, which protects the caller's data, and the copy is then sorted. The sort is the dominant cost, taking time proportional to n log n, while the scoring takes time proportional to n multiplied by the cost of the checks.

A reader implementing the algorithm must pay attention to the comparison function passed to the sort. A comparison that subtracts one score from another produces the descending order when the second score is subtracted from the first or the ascending order in the opposite case, and mistakes in this expression are a common source of reversed results. It is also necessary to treat a missing query as an empty one so that the title test is skipped. Tests should include a list of events with all possible combinations of the three signals, so that every score from zero to nine that can occur is observed and the resulting order is checked.

4.4.5 Save a Favourite

1. Read eventoraFavorites from local storage; if it cannot be parsed, use an empty list.
2. Check whether the selected event is already present by identifier.
3. If present, remove it; otherwise add it to the favourites list.
4. Update the page state with the new list.
5. Serialize the new list and save it to eventoraFavorites.

This procedure implements a toggle: the same user action either adds or removes a favourite depending on the current state. Reading the stored list and falling back to an empty list if it cannot be parsed protects the page against corrupted data. The check for presence by identifier ensures that an event cannot appear twice in the list, which preserves the invariant that every event occurs at most once among the favourites. Updating the page state before or together with saving the list guarantees that the screen and the storage agree.

The procedure takes time proportional to the length of the favourites list, since checking presence and removing an item both require a scan. For the small lists that a person is likely to keep, this is negligible. Testing should confirm that the first toggle adds, the second removes, that the list survives a page reload and that an empty or damaged stored value does not stop the procedure from working.

4.4.6 Create a Custom Event

1. Collect the event details from the form.
2. Validate required fields in the user interface.
3. Read customEvents from local storage, using an empty list if parsing fails.
4. Construct a new event record with an identifier and the entered event details.
5. Place the new record at the start of the stored list.
6. Write the updated list to local storage and navigate back to Explore.

The creation of a custom event consists of collecting, validating, storing and navigating. Validation in the interface is the first line of defence, and it should check that required fields such as the title, the date and the location are present and that the price is a valid number. The construction of the record with an identifier is the step that deserves the most care, because the identifier must differ from those of the built-in events and of earlier custom events. A common approach is to use a timestamp or a random value, and any such scheme should be tested for collisions.

Placing the new record at the start of the stored list means that the newest custom events are returned first when the data function combines the lists, so a user who has just created an event can find it at the beginning of the custom portion of the catalogue. Writing the list and then navigating to Explore completes the operation, and the user sees the new event among the others because the data function reads

the storage when the page is prepared. Since the data remains in the browser, the event is visible only to the person who created it, which should be made clear to users of the prototype.

5. RESULTS

5.1 Implemented Features

The source archive contains code for the following functional areas. This table is a source-inspection inventory, not a claim that every feature has passed a formal acceptance test in a deployed environment.

Feature area	Evidence in source archive	Result description
Multi-page navigation	AppRoutes.jsx declares page routes.	Separate task pages are connected through React Router.
Sample event catalogue	src/data/events.js defines eight events.	Built-in event information is available to the front end.
Search and filters	Explore.jsx and algorithm utilities.	Search, category selection and sort state are represented in the interface.
Favourites	Favorites.jsx and Explore.jsx use eventoraFavorites.	Favourite records can be retained in local browser storage.
Custom events	CreateEvent.jsx and getAllEvents().	Custom records can be saved and combined with sample events.
Booking and tickets	Booking.jsx and MyTickets.jsx use tickets.	A local booking/ticket flow is implemented in the UI.
Account interface	Login.jsx and Register.jsx use users.	Prototype account records are browser-stored.
Recommendation logic utilities	ranking.js and similarity.js.	Weighted ranking and token similarity functions are implemented.
Approximate matching	fuzzySearch.js.	A Levenshtein-distance-based fuzzy matcher is implemented.
Calendar and recommendation pages	Calendar.jsx and Recommendation.jsx.	Dedicated interfaces exist for date browsing and recommendations.

The inventory of features in the table is based on inspection of the source archive. For each feature area, the middle column names the files in which the evidence appears and the right column describes what the evidence shows. This format allows a reader to check each statement against the source and to see that the claims are limited to what is visible there. A feature is listed as implemented when the source contains the interface elements and the logic that would allow the behaviour, and it is not claimed to have passed an acceptance test.

Taken together, the features cover the whole user journey from arrival to ticket. The navigation feature connects the pages, the sample catalogue supplies the content, search and filters let the user narrow it, favourites let the user keep a

shortlist, custom events let the user contribute, booking and tickets let the user conclude an attendance, and the account interface frames the experience for a named user. The algorithm utilities support the discovery features, and the calendar and recommendation pages offer alternative ways of reaching an event. The coverage is broad for a course project, and its depth is the subject of the remaining sections.

It is also useful to mention what the feature inventory does not include. It does not list an organiser dashboard with real attendance figures, because the source provides a dashboard view without a live data source. It does not list email or message confirmations, a payment receipt from a gateway, a map of venues or a reminder service. These omissions are consistent with the scope and with the limitations table, and by naming them the report avoids the impression that the feature list is complete in the sense of a commercial product.

5.2 Algorithm Walkthroughs

Example 1: Pattern matching. Suppose the query is “tech” and an event record contains the title “Hyderabad Tech Summit”. The algorithm converts both the query and combined searchable text to lowercase. Since the lowercase title contains “tech”, the event satisfies the pattern check. A query can also match the category or location because all three fields are concatenated before matching.

Example 2: Fuzzy matching. Suppose a user types a short misspelling of a word in an event title. The fuzzy matcher first checks for a direct substring match. If that fails, it calculates edit distance between the query and each word in the text. For a query length of six characters, the implementation allows a distance of at least one and otherwise floor(6/3), which is two. This can tolerate a small number of character edits, although the threshold can also produce false positives for short queries.

Example 3: Ranking. A featured event begins with a score of two. If it is also in favourites, three more points are added. If its title contains the query, four more points are added. The maximum score from these three rules is nine. A non-featured event that is neither favourited nor a title match receives zero. The result

is sorted in descending score order. This example illustrates a simple, deterministic ranking strategy.

Example 4: Similarity. If input A contains the unique tokens {music, live, festival} and input B contains {live, festival, outdoor}, the intersection contains two tokens and the union contains four. The similarity score is therefore 2/4 = 0.5. This is a mathematical illustration of the function's set-based behavior; it is not a measured recommendation accuracy result.

Example 5: Fuzzy matching with a single-character error. Consider the query sumit, which is a misspelling of a word in the title Hyderabad Tech Summit. The query is first lowercased, and a direct substring test fails, because the lowercase text contains summit and not sumit. The text is split into the words hyderabad, tech and summit. The query has five characters, so the tolerance is the larger of one and the floor of five divided by three, which is one. The distance between sumit and summit is one, since inserting the second letter m into sumit produces summit. The distance is within the tolerance, so the function returns true and the event is accepted despite the missing letter. The words hyderabad and tech are also compared, but a single successful comparison is enough.

Example 6: A transposition that is not accepted. Consider the query tehc, in which the last two letters of the word tech are exchanged. The direct substring test fails. The query has four characters, so the tolerance is one. The Levenshtein distance between tehc and tech is two, because the strings differ in the last two positions and no single insertion, deletion or substitution can repair both. Since two exceeds the tolerance of one, the function returns false for the word tech and for the other words, and the event is not found. This example shows a limit of the implementation. An approach that counted a transposition as one edit would accept the query, and this is the improvement suggested in the discussion of the Damerau variant.

Example 7: Pattern matching across fields. Suppose the query is hyd. The pattern-matching function concatenates the title, category and location of each event, converts the result to lowercase and looks for the pattern. The event whose title is Hyderabad Tech Summit matches through its title, and the same event also matches through its location, which contains Hyderabad. An event held in another city does not match unless one of its fields contains the three letters. If the query

were music, an event in the Music category would match through its category field even if the title did not contain the word. The example shows that concatenation lets a single query reach several fields.

Example 8: Ranking with several events. Suppose four events are ranked for the query summit. Event A is featured, is a favourite and has a title containing the query, so its score is two plus three plus four, which equals nine. Event B is featured only, with a score of two. Event C is not featured and is not a favourite but has a title containing the query, with a score of four. Event D is a favourite only, with a score of three. The sorted order is A, C, D and B. The example shows that a title match outweighs promotion and favourite membership taken singly, and that the three signals can be combined. If two events had the same score, the stable sort would keep them in their original order.

Example 9: Similarity with a different word order and an extra token. Let the first text be Hyderabad Tech Summit and the second text be Tech Summit Hyderabad 2026. The first set contains hyderabad, tech and summit, and the second contains tech, summit, hyderabad and 2026. The intersection has three elements and the union has four, so the similarity is three divided by four, which is 0.75. The different order of the words has no effect. If the second text were written with a trailing comma after summit, the token summit, would differ from summit, the intersection would shrink to two elements and the union would grow to five, and the similarity would fall to 0.4. This illustrates the sensitivity to punctuation mentioned in the design chapter.

These walkthroughs are not experimental results. They are hand-traced applications of the rules described in the source, and they are provided to make the behaviour of the algorithms concrete and to suggest test cases. Their value is that each can be reproduced by anyone who has the rules and the example input, and each can be turned into a unit test whose expected value is already known. In a later evaluation, the same cases can be run against the real functions to confirm that the implementation behaves as the hand calculation predicts.

5.3 Verification Checklist

A practical verification pass should be performed in a browser after installing the dependencies. The following checklist describes the checks appropriate for the

supplied prototype. The report does not claim that these tests were executed automatically or that all passed.

Test ID	Action	Expected observation
T-01	Open the root route.	Home page loads without a route error.
T-02	Open Explore and browse event cards.	Built-in sample events are visible.
T-03	Search for a word in an event title.	Matching events are displayed according to the page's search behavior.
T-04	Change the category filter.	Visible results are narrowed to the selected category.
T-05	Mark an event as favourite and refresh.	The saved event remains in the browser's Favorites list if local storage is retained.
T-06	Open an event detail route.	The selected event's information is displayed.
T-07	Complete the booking interface with valid sample data.	A ticket record is stored locally and the user is directed to My Tickets.
T-08	Create a custom event and return to Explore.	The custom event appears alongside built-in events.
T-09	Register a test account and try login.	The prototype account flow uses the browser-stored users list.
T-10	Clear local storage and reload.	Locally stored favourites, custom events, tickets and user records may no longer be available.

The checklist in the table is a minimal functional test plan. Each row names an action and an expected observation, so that a tester can follow it without special knowledge of the code. The tests T-01 and T-02 confirm that the application starts and displays the catalogue. T-03 and T-04 test search and filtering, T-05 tests persistence of favourites, T-06 tests routing to an event, T-07 tests the booking flow, T-08 tests the creation of a custom event, T-09 tests the account flow and T-10 tests the effect of removing browser storage. Together they exercise each of the principal modules and each of the browser storage keys.

A good test plan includes boundary and negative cases in addition to the normal cases. For search, a tester should try an empty query, a query consisting only of spaces, a query in capital letters, a query with a misspelling and a query that matches nothing. For booking, the tester should try submitting with missing fields, with a quantity of zero or a negative number, and with an invalid email address, to see how the interface responds. For routes, the tester should open an event address with an identifier that does not exist. For storage, the tester should place invalid text under a key and reload to check that the guarded parsing prevents a failure. These cases often reveal weaknesses that normal use does not.

Recording the results is as important as running the tests. A simple table with the test identifier, the date, the browser and version, the observed result and a pass or

fail mark gives the project an auditable record. Failures should be described with the steps needed to reproduce them, so that a developer can find and correct the cause. Screenshots of key screens can be added to the report as numbered figures after the tests have been run, and the figure list can then be updated accordingly. The report does not claim that these steps have already been performed.

Automated testing would complement the manual checklist. The algorithm functions are suitable for unit tests, because they require no browser. A test runner can call each function with prepared inputs and compare the output with expected values, such as the hand-traced results of the walkthroughs, and it can run in seconds after every change. Component tests can render a page, simulate a click or a keystroke and check what appears, and end-to-end tests can drive a real browser through a complete journey from search to ticket. Adding these layers would convert the checklist from a one-time activity into a continuous safeguard.

5.4 Source-Based Outcome

The main outcome is a modular event-management front end with multiple user workflows and a set of explicit algorithm utilities. The separation of pages and utilities provides a foundation for further development and makes the algorithm logic independently inspectable. The use of local storage reduces setup requirements for demonstration but is not a substitute for a database, secure account service or payment integration.

Quantitative performance and accuracy conclusions cannot be made from the source archive alone. To report search precision, recommendation relevance, page-load time, booking success rate or accessibility scores, the team should define test cases, run the application in a controlled environment and record actual measurements. Screenshots of the running application may be inserted as numbered figures after those checks are completed.

The principal outcome is that the objectives set out in the first chapter can be matched with identifiable parts of the source. The unified interface corresponds to the set of pages and shared components. The organization into reusable pieces corresponds to the component structure. Search, filtering and sorting correspond to the state variables of the Explore page and to the algorithm utilities. Favourites,

tickets and custom events correspond to the storage keys and the pages that use them. The demonstration of algorithms corresponds to the four files in the algorithm directory, and route-based navigation corresponds to the route table. The final objective, honest documentation, corresponds to the structure of this report.

The outcome also points to what should happen next. The prototype is ready for a verification pass, which would convert the statements in this chapter from inspection-based claims into observed results. After that, the highest-value additions would be a server for shared data and authentication, automated tests for the algorithms and an evaluation of search and ranking with realistic queries. The presence of a clear boundary between the front end and the logic makes it possible to take these steps one at a time without rewriting the application, and this is perhaps the most important structural benefit of the design.

6. CHALLENGES AND LIMITATIONS

6.1 Issues Faced During Development

One design challenge in a multi-page application is keeping the same event record usable across listing, detail, favourite, booking and calendar views. A consistent event object reduces mismatches between these pages. The supplied event data structure uses common fields for the built-in catalogue, while custom events are merged from browser storage.

A second challenge is search quality. Exact matching is fast and understandable but can miss misspellings. The fuzzy matcher addresses some spelling variation through edit distance, but its threshold is heuristic and may accept unrelated words, particularly for short queries. The token similarity function also uses simple whitespace tokenization and does not remove punctuation, so it is not a full natural-language understanding system.

A third challenge is browser-side persistence. localStorage is easy to use and works without a backend, but it is scoped to the browser profile, has limited storage capacity, can be cleared by the user and can be modified by client-side scripts. It should not be used as a secure source of truth for passwords, payments or inventory.

Another implementation consideration is separating interface demonstrations from live services. Booking and payment screens can demonstrate user flow, but a production service needs verified inventory, server-side booking confirmation, payment-provider callbacks and failure handling. A backend package manifest is present in the archive, yet a fully connected API implementation cannot be established from the reviewed source files.

Managing state across many pages was a recurring difficulty. When the same information, such as the list of favourites, is needed by the Explore page, the Favorites page and possibly the detail page, there are two choices: keep it in a shared location or let each page read it from storage. The project follows the second approach, which works but requires every page to read and write the same key in the same format. A mismatch in format, such as one page storing identifiers and another storing complete records, would produce confusing behaviour. The team had to agree on the shape of each stored value and apply it consistently, and this agreement is a form of interface between modules even though it is not written as code.

Handling dates required attention to detail. The event dates are written in the year-month-day format, which sorts correctly as text, but a calendar view needs to convert them into days of a month and to compare them with a selected date. Conversion between strings and date objects can introduce errors related to time zones and to the numbering of months, which begins at zero in JavaScript. The calendar page must also decide how to treat events that have already passed, and how to display days with several events. These details are easy to underestimate when a feature is described as a calendar, and they illustrate how a small requirement can conceal considerable logic.

Designing the forms was another source of effort. The booking form collects several kinds of information, and each has its own rules: a name should not be empty, an email should have a recognisable structure, a phone number should contain the right number of digits, and a UPI identifier follows its own pattern. The payment choice determines which fields are relevant, so the form changes according to the selection. Keeping the form state consistent while fields appear and disappear, and presenting clear messages when input is invalid, required careful organisation of the state and of the handlers.

Making the layout work on different screens also took time. Event cards, filter panels, forms and ticket views have to be arranged differently on a wide desktop display and on a narrow phone. Flexible grids and media queries help, but they must be tested, and a layout that looks acceptable at one width may overflow or compress at another. Because testing on many devices is time-consuming, the team treats responsive verification as a task to be completed before final submission and not as something already confirmed.

6.2 Solutions Applied

- Separate pages by user task and declare their paths centrally in AppRoutes.jsx.
- Represent built-in events using a consistent object structure.
- Create reusable components for navigation, cards, search and ticket presentation.
- Keep matching and ranking logic in standalone utility files.
- Use lowercasing for case-insensitive text comparison.
- Use a dynamic-programming matrix to calculate Levenshtein edit distance.
- Use local-storage parsing guards in data and page modules where shown in the source.
- Keep the prototype's limitations explicit so that local UI behavior is not mistaken for a verified external service.

Dividing the application into pages by task, each with its own file, reduced the complexity of every individual module. A page that deals with only one task has a limited set of state variables and handlers, and a developer can understand it without reading the others. Declaring the routes in a central file complements this division, because the relationship between addresses and pages is visible in one place. The cost of this solution is a larger number of files, but the benefit in clarity is considerable.

A consistent event object solved the problem of different pages disagreeing about what an event is. By defining the fields once in the data module and requiring custom events to supply the same fields, the project ensured that any component that renders an event can rely on the presence of a title, a category, a date, a location, a price, a description, an image and a featured flag. The reusable components for cards, search and tickets build on this consistency, and they allow

improvements, for example in the layout of a card, to be made in one file and to appear everywhere.

Keeping the algorithms in standalone files solved two problems. It prevented the pages from becoming cluttered with string-processing code, and it made the algorithms available for study and testing in isolation. The dynamic-programming matrix for edit distance, lowercase normalisation and the set-based similarity are each contained in a small function with a clear purpose. A reader who wishes to understand the search behaviour can read these files without having to understand the interface code.

Guards around the parsing of stored data solved the problem of fragile persistence. A try-catch block around the conversion from text to data ensures that a corrupted value results in an empty list and not in a failed page. Together with the use of default values when a key is absent, this makes the application robust against the usual hazards of browser storage, such as a first visit with no data or data written by an earlier version. The approach is not a substitute for validation, but it is an effective and inexpensive safeguard.

Making the limitations explicit solved the problem of overstating the work. By placing the limitations in a table with the effect and the recommended improvement for each, the report gives the reader a realistic picture and the team a to-do list. This practice also served the course objective of responsible technical writing, in which claims are matched with evidence, and it helped avoid a situation in which a grader or user could mistake the interface of a booking page for a verified service.

6.3 Limitations of the System

Limitation	Effect	Recommended improvement
Local-only storage	Data is tied to one browser and can be deleted.	Use a database and authenticated server API.
Prototype authentication	Browser-stored account records are not secure authentication.	Hash passwords on the server, use secure sessions/tokens and validate requests.
No confirmed payment gateway	Payment UI alone cannot prove a real transaction.	Integrate a provider with server-side verification and test/sandbox transactions.
Static sample event data	Built-in records are illustrative and do not guarantee current event availability.	Connect to a maintained event database or verified provider API.

Heuristic ranking	Fixed weights do not learn user preferences and may produce ties.	Define relevance signals, tune weights on test data and evaluate results.
Basic text normalization	Punctuation and language-specific variants may reduce matching quality.	Normalize punctuation, tokenize more robustly and support multilingual search.
No formal benchmark in archive	Performance and recommendation accuracy are not measured.	Create repeatable test datasets and report measured results.
Backend not confirmed	A backend package manifest exists but no complete API contract is verified.	Implement and document routes, schemas, validation, deployment and error handling.

The first group of limitations concerns data. Because the favourites, custom events, tickets and users are stored in the local storage of the browser, they are tied to one browser profile on one device. A user who opens the application on a phone after using it on a laptop will find none of the earlier data, and a user who clears the browsing data will lose it all. Storage also has a limited capacity, and although the amount of data in the prototype is far below the limit, a large number of custom events or tickets with images could approach it. The sample events are fixed and do not reflect real availability, which means that a displayed price or date is illustrative.

The second group concerns security and privacy. The booking interface collects personal information, including a name, an email address, a phone number and a UPI identifier, and the prototype stores ticket records in the browser. Anyone who has access to the browser, or any script that runs on the same origin, can read those values. The registration interface stores user records in the same way, and the lack of password hashing and server-side checks means that it cannot protect an account. From the perspective of privacy, collecting and storing such data requires a clear purpose, minimisation and protection, and a production service would have to comply with the relevant data-protection rules. For the prototype, the safest practice is to use only fictitious data during demonstration.

The third group concerns the quality of search and ranking. The pattern matching considers only the title, category and location, the fuzzy matcher treats a transposition as two errors and its threshold may accept unrelated short words, and the similarity function does not remove punctuation or reduce words to their roots. The ranking function has fixed weights, produces ties and does not account for the date, the price or the preferences of the user. None of these weaknesses prevents the demonstration of the algorithms, but they would matter in a real search

service, where relevance has a measurable effect on how many users find what they want.

The fourth group concerns testing and quality assurance. The archive does not include automated tests, a performance benchmark or an accessibility audit, and so the report cannot state test coverage, response times or compliance with accessibility guidelines. Cross-browser behaviour and responsive layout are also unverified. This limitation is the easiest to remove, because it requires effort and not new technology, and it is the first item on the recommended list of improvements.

The fifth group concerns services that the prototype does not implement. There is no confirmed server, no verified payment gateway, no email or message notification and no shared inventory, and so features that depend on them, such as preventing the overbooking of an event or sending a confirmation to a buyer, are not possible. The wildcard route that returns the home page and the absence of an explicit not-found page are smaller limitations of the same kind. The table of limitations summarises the effects and remedies, and the future work in the next chapter organises them into a plan.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

Eventora demonstrates a structured approach to event discovery and booking-oriented interaction through a React web application. Its source includes dedicated pages for exploring events, viewing details, saving favourites, creating custom events, browsing a calendar, handling booking forms and viewing tickets. Route-based navigation and reusable components organize the user interface into manageable parts.

The project also connects to the academic focus of Data Structures and Algorithms - 3 through explicit text-processing utilities. Pattern matching provides case-insensitive substring search; fuzzy matching uses Levenshtein distance to allow a bounded number of edits; token similarity calculates a Jaccard-style overlap score; and ranking combines featured status, favourite membership and

title matching into a weighted score. These techniques are simple enough to explain and extend, and they show how algorithm design can support practical application features.

The strongest conclusion supported by the submitted archive is that a front-end prototype and its core algorithm utilities have been implemented in source form. A live production service, secure authentication, persistent remote database and real payment processing are not established by the reviewed code. The next development stage should focus on validating the running application and moving sensitive and shared operations to a secure backend.

The project has shown that a reasonably complete event discovery interface can be built with a small set of tools and a clear structure. React supplied the component model, Vite the build environment, React Router the navigation and Lucide the icons, and the combination allowed the team to concentrate on the logic of the application and not on infrastructure. The division into pages, components, data and algorithms made the work manageable for a team and the result understandable for a reader.

From the perspective of the course, the most valuable outcome is the connection between data structures and a visible feature. The array, the object, the set and the matrix are not abstract topics in this project but concrete parts of the search and ranking behaviour that a user sees. The Levenshtein routine shows how dynamic programming solves a problem that would otherwise require an exponential search over edit sequences, the Jaccard measure shows how a set operation can express similarity, and the ranking function shows how a simple weighted score can order a list. Each of these is analysable in terms of time and space, and the analysis has been included so that the connection to the syllabus is explicit.

The conclusion must also be stated with restraint. The work is a prototype whose behaviour has been described from the source and not measured in a live environment, and its persistence, authentication and payment features are not production-grade. The report has attempted to say this clearly at every stage, because a clear statement of limits is a part of the quality of the work and not a weakness in it. The next stage of development should therefore begin with verification and then proceed to a secure back end, so that the strengths of the present design can be carried into a service that serves real users.

7.2 Future Work

- Build a documented Express API with routes for events, users, favourites and bookings.
- Connect a database to store event records, users and tickets across devices.
- Implement secure authentication with server-side password hashing, validation and session management.
- Integrate a payment provider in test mode first, then add verified transaction handling and booking confirmation.
- Replace sample events with a maintained data source and validate event dates, prices and availability.
- Improve search by normalizing punctuation, using robust tokenization and searching across descriptions as well as titles, categories and locations.
- Extend recommendations with explicit user preferences, categories followed, event interactions and transparent relevance explanations.
- Evaluate ranking using a labelled dataset and measures such as precision at k, recall at k and user feedback.
- Add automated unit tests for the algorithm utilities and component-level tests for forms and navigation.
- Run accessibility checks, responsive layout tests, security review and performance measurements.
- Provide error states for unavailable events, invalid form data, storage failures and booking failures.
- Deploy the application and document environment variables, build steps and operational monitoring.

The improvements listed above can be arranged into three stages that follow the order in which they would be undertaken. The first stage is verification and hardening of the existing prototype. It consists of running the checklist in a browser, writing unit tests for the four algorithm files, auditing accessibility and responsive layout, adding validation and error messages to the forms, adding a not-found page and improving the handling of storage failures. These tasks do not require new infrastructure and would raise confidence in the current behaviour, and they should be completed before further features are added.

The second stage is the introduction of a back end. An Express service would expose a documented set of routes for events, users, favourites and bookings, and a database would hold the records so that they are available on every device. Authentication would be implemented with server-side password hashing and secure sessions or tokens, and every request would be validated. The front end would be modified to fetch data from the interface, show loading and error states and send bookings to the server. During this stage, the sample events would be replaced by a maintained data source, and the identifiers, dates and prices would be validated so that the catalogue reflects real events.

The third stage is the extension of the intelligence of the system. Search can be improved by normalising punctuation, using stronger tokenisation, searching the description and supporting transposition errors through the Damerau variant of edit distance. Ranking can be extended by adding signals such as the date, the price, the category preferences of the user and the popularity of an event, with weights tuned on a set of labelled examples and evaluated with measures such as precision and recall at a fixed number of results. Explanations of why an event is recommended, for example because it matches a followed category, would make the recommendations transparent to the user.

Payment integration deserves its own consideration. It should be introduced in the sandbox mode of a provider, with the server creating the payment request and verifying the result, and the booking record should carry a status such as pending, paid, failed or cancelled. Only after the full cycle, including failure and refund paths, has been tested should a live mode be considered. Notifications by email or message can be added once bookings are stored on a server, and they require attention to consent and to the protection of contact details.

Operational concerns complete the roadmap. The application should be deployed to a hosting service with a documented build procedure and environment variables, and the deployment should be monitored for errors and slow responses. A continuous integration process that runs the linter and the tests on each change would protect the quality of the code. Documentation for developers, containing the setup steps, the structure of the data and the contract of the interface, would allow others to contribute. These steps convert a student prototype into a

maintained product, and they provide natural directions for further academic work.

7.3 Skills Learned During Development

The project provides practice in decomposing a user-facing problem into separate modules, building React function components, managing state with hooks, connecting screens through routes and designing reusable UI elements. It also gives practical experience with JavaScript arrays and objects, browser local storage, event-data modeling and CSS-based interface styling.

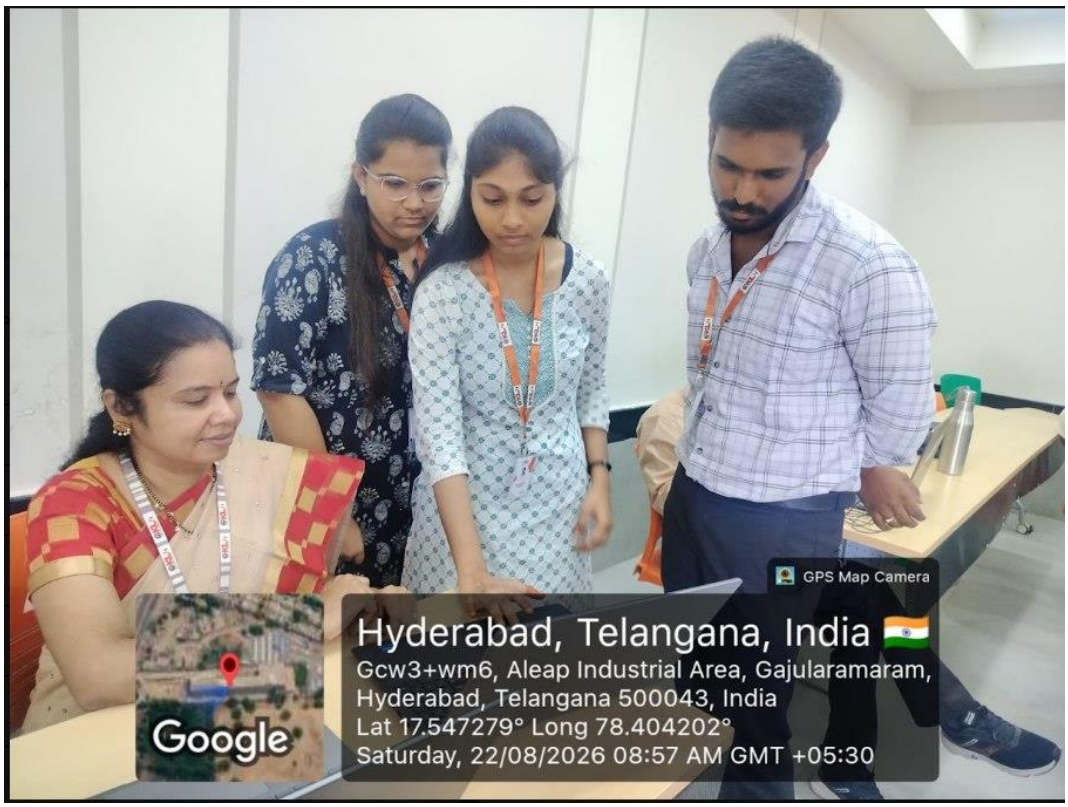
The algorithm modules reinforce string manipulation, set operations, dynamic programming, edit-distance computation and score-based sorting. Writing pseudocode and tracing example inputs helps explain the connection between implementation and expected behavior. Finally, documenting limitations encourages responsible technical reporting: features should be described according to the code and tests that actually support them.

On the technical side, the team gained practical experience of the component-based style of user-interface development. Understanding how state drives rendering, how effects connect components to the outside world and how routes divide an application into views are skills that apply to many frameworks, not only to React. Working with Vite and npm taught the team the workflow of a modern front-end project, including the use of a lock file, scripts for development and build, and linting. Reading and writing JSON, handling errors in parsing and designing a consistent data model taught lessons about the interface between programs and data.

On the algorithmic side, the project turned familiar topics into working code. Implementing the edit-distance matrix required careful attention to indices and base cases, and tracing it by hand on examples built an intuition for how the table is filled. Working with sets for similarity strengthened the understanding of intersection and union and of how a data structure can express a mathematical definition. Implementing a ranking with a stable sort and an immutable copy showed why it matters to know how library functions behave. Analysing the cost of each algorithm reinforced the habit of asking how a solution scales.

On the professional side, the team practised collaboration, planning and documentation. Dividing the work by module, agreeing on shared data formats and reviewing each other's work are skills that are required in every software team. Writing the report taught the discipline of matching statements with evidence, of presenting requirements, design, implementation and results in a logical order, and of stating limitations frankly.

Geo Tag:



REFERENCES

[1] React, “React Documentation.” Available: https://react.dev/ (accessed 11 October 2026).

[2] Vite, “Vite Guide.” Available: https://vite.dev/guide/ (accessed 11 October 2026).

[3] React Router, “React Router Documentation.” Available: https://reactrouter.com/ (accessed 11 October 2026).

[4] Lucide, “Lucide Icons.” Available: https://lucide.dev/ (accessed 11 October 2026).

[5] MDN Web Docs, “Window: localStorage property.” Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage (accessed 11 October 2026).

[6] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, pp. 707–710, 1966.

[7] G. Salton and M. J. McGill, Introduction to Modern Information Retrieval. New York: McGraw-Hill, 1983.

[8] Eventora project source archive supplied for this report: eventora_favorites_added (1).zip, including React pages, algorithm utilities, event data and package manifests.

In-text citation examples: component-based UI and routing are implemented with the libraries listed in [1]–[4]. Browser-local persistence uses the localStorage API described in [5]. The edit-distance concept used by the fuzzy matcher is associatedwith [6]. The token-overlap approach is related to basic information-retrieval and set-similarity concepts discussed in [7].